lamae

R3.05 · TD-TP 06

PROGRAMMATION SYSTÈME

Les processus

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Le LAMAE élève une ruche numérique. La reine (le processus père) pond des ouvrières avec fork, les envoie au travail avec exec et doit recueillir chacune d'elles avec wait, sinon la ruche se remplit de zombies. Ta mission : faire tourner la ruche, puis construire « lamash », le mini-shell du laboratoire.

OBJECTIFS DE LA SÉANCE

- ☐ Distinguer **programme** et **processus** ; observer les processus avec `ps`, `pstree` et `/proc`.
- ☐ Créer des processus avec `fork` et distinguer le père du fils grâce à la valeur de retour.
- ☐ Terminer un processus avec `exit` et récupérer son code de retour avec `wait` / `waitpid` (`WIFEXITED`, `WEXITSTATUS`).
- ☐ Reconnaître et éviter les **zombies** ; comprendre ce qu'est un **orphelin**.
- ☐ Remplacer le programme d'un processus avec la famille `exec`, et combiner **fork + exec + wait** comme le fait le shell.
- ☐ Programmer de façon sûre : tester `fork() == -1`, borner la création de processus, ne jamais passer une saisie utilisateur à `system()`.

FIL ROUGE Opération « La Ruche »

Chaque mission rapporte un **drapeau** `FLAG{...}` à noter dans `badge.txt`. Les exercices sont cotés de ★ (incontournable) à ★★★ (pour aller loin). Une règle absolue dans la ruche : **toute ouvrière créée doit être recueillie** (`wait`), et **aucune boucle de `fork` n'est lancée sans borne**. Une ruche qui pond sans limite, c'est une *fork bomb* : elle paralyse la machine de toute la salle.

Mission	Contenu	Durée
0 · Repérage	`ps`, `pstree`, `/proc`, `getpid`, `getppid`	10 min
1 · La ponte	`fork`, père et fils, énigme des `fork` en série	25 min
2 · Vie et mort	`sleep`, `exit`, `wait`, zombies et orphelins	25 min
3 · L'essaim	n fils, `waitpid`, codes de retour, fichier partagé	20 min
4 · La métamorphose	La famille `exec`	15 min
5 · lamash	Mini-shell : fork + exec + wait	25 min
Bonus	Pour les rapides : `&`, `cd`, ramasse-zombies	—

COMPILATION

Les fonctions système (`fork`, `getpid`, `waitpid`...) ne font pas partie du C standard mais de la norme **POSIX**. Avec `-std=c17`, placez cette ligne **avant tout `#include`** pour les rendre visibles :

```c
#define _POSIX_C_SOURCE 200809L   // active les déclarations POSIX.1-2008
#include <stdio.h>
#include <stdlib.h>      // exit, EXIT_SUCCESS, EXIT_FAILURE
#include <unistd.h>      // fork, getpid, getppid, sleep, exec...
#include <sys/types.h>   // pid_t
#include <sys/wait.h>    // wait, waitpid, WIFEXITED, WEXITSTATUS
```

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g ruche.c -o ruche
```

Rappel — Les primitives du sujet

Cette section rappelle les primitives de manipulation des processus (voir le CM 03a). Toutes, sauf `exit`, sont des **appels système** : le processus passe en mode noyau pour les exécuter.

Primitive	Rôle
`pid_t fork(void)`	Crée un processus fils, **copie** du père (code, données, pile, tas, descripteurs ouverts). Renvoie **0 dans le fils**, le **PID du fils dans le père**, et **-1** en cas d'échec (aucun fils n'est alors créé).
`unsigned sleep(unsigned s)`	Le processus appelant s'endort pendant `s` secondes (état S).
`pid_t wait(int *statut)`	Attend la fin de **n'importe lequel** de ses fils et renvoie son PID. Si un fils est déjà terminé (zombie), retour immédiat. Sans fils : renvoie -1 (`errno = ECHILD`). Passez `NULL` si le statut ne vous intéresse pas.
`pid_t waitpid(pid_t pid, int *statut, int options)`	Attend **un fils précis** (ou n'importe lequel si `pid = -1`). Option `WNOHANG` : ne bloque pas.
`void exit(int code)`	Termine le processus : vide les tampons de `stdio`, ferme les fichiers, transmet `code` au père. Les instructions qui suivent ne sont jamais exécutées.
`pid_t getpid(void)` / `pid_t getppid(void)`	Renvoie le PID du processus appelant / celui de son père.
`execlp`, `execvp`, `execv`…	Remplace le programme du processus par un autre (même PID). Ne revient **qu'en cas d'échec**.

> **PIÈGE CLASSIQUE** **Le fils n'est pas « totalement identique »**
>
> Après `fork`, père et fils ne diffèrent pas seulement par la valeur renvoyée par `fork` : le fils a **son propre PID**, un **PPID** différent (le PID du père), des compteurs de temps CPU remis à zéro, aucun signal en attente, et n'hérite pas des verrous de fichiers du père (voir `man 2 fork`). Et surtout, après le `fork`, **les deux mémoires sont séparées** : modifier une variable dans le fils ne change pas celle du père.

Mission 0 — Repérage dans la ruche

Un **programme** est un fichier (une recette) ; un **processus** est un programme en cours d'exécution (la recette en train d'être préparée), avec son PID, sa mémoire, ses fichiers ouverts et son propriétaire.

EX 0 Observer les processus ★ 10 min

```
• • • TERMINAL
$ echo $$                       # PID du shell courant
$ ps -l                         # processus du terminal : état S, PID, PPID...
$ pstree -p $$                  # l'arbre à partir du shell
$ ps -ef | head -5              # tous les processus du système
$ grep -E 'Name|State|PPid' /proc/$$/status
$ ls -l /proc/$$/fd             # les descripteurs ouverts du shell
```

1. Quel est le PID de votre shell ? Quel est son père (PPID) ? Remontez la filiation jusqu'au PID 1 avec `ps -o pid,ppid,cmd -p <pid>`.
2. Lancez `sleep 100 &` puis `ps -l` : dans quel état est `sleep` ? Que signifie la lettre ? Terminez-le avec `kill %1`.
3. Dans `/proc/$$/fd`, quels sont les descripteurs 0, 1 et 2 ? Vers quoi pointent-ils ?

Drapeau : `FLAG{` + la lettre d'état d'un processus endormi + `}`.

Mission 1 — La ponte : fork

EX 1 La reine et sa première ouvrière (énoncé 1.1) ★ 10 min

Écrivez `famille.c` : après un `fork`, chacun des deux processus affiche son identité.

```
je suis le père, mon PID est xxxxx, le PID de mon père est zzzzz
je suis le fils, mon PID est yyyyy, le PID de mon père est xxxxx
```

```
• • • famille.c (squelette)
pid_t pid = fork();
if (pid == -1) {                // TOUJOURS tester l'échec
    perror("fork");
    exit(EXIT_FAILURE);
}
if (pid == 0) {
    /* code du fils */
} else {
    /* code du père : pid contient le PID du fils */
}
```

1. Qui est le père du processus père (zzzzz) ? Vérifiez avec `echo $$`.
2. Lancez le programme plusieurs fois. L'ordre des deux messages est-il toujours le même ? Qui le décide ?
3. Ajoutez une variable `int miel = 10;` déclarée **avant** le `fork`. Le fils fait `miel += 100`, le père `miel += 1`, puis chacun affiche `miel` **et** son adresse `&miel` (format `%p`). Les adresses sont identiques, les valeurs différentes : comment est-ce possible ?

> **INDICE**
>
> Ce sont des adresses **virtuelles** : chaque processus a son propre espace d'adressage. Le noyau partage d'abord les pages entre père et fils, puis copie une page dès que l'un des deux la modifie (*copy-on-write*).

SÉCURITÉ Tester fork() == -1

`fork` peut échouer : table des processus pleine, limite `RLIMIT_NPROC` atteinte, mémoire insuffisante. Un programme qui ignore ce cas prend `-1` pour un PID de fils… et un `kill(pid, SIGTERM)` ultérieur avec `pid = -1` enverrait le signal **à tous vos processus** ! Règle ANSSI : toujours vérifier la valeur de retour des appels système.

EX 2 L'énigme des trois fork

★★ 10 min

Sans exécuter, prédisez le nombre de lignes affichées par chacun de ces programmes, puis vérifiez. Dessinez l'arbre des processus du programme B.

programme A

```c
int main(void) {
    fork();
    fork();
    fork();
    printf("bzz %d\n", getpid());
    return 0;
}
```

programme B

```c
int main(void) {
    for (int i = 0; i < 3; i++) {
        if (fork() == 0)
            printf("ouvrière i=%d pid=%d ppid=%d\n", i, getpid(), getppid());
    }
    return 0;
}
```

1. Combien de processus au total pour A ? pour B ? Quelle est la règle pour n `fork` exécutés à la suite ?
2. Dans B, ajoutez `sleep(30);` avant le `return`, lancez le programme en arrière-plan (`./b &`) et observez l'arbre avec `pstree -p $$`.
3. Modifiez B pour que **seule la reine** ponde : 3 ouvrières exactement, et aucune ouvrière ne pond. Quelle instruction ajouter dans le fils ?

Drapeau : `FLAG{` + le nombre de « bzz » affichés par A + `}`.

EX 3 Le message fantôme

★★ 5 min

Testez ce programme, puis relancez-le en redirigeant la sortie : `./fantome > sortie.txt ; cat sortie.txt`.

fantome.c

```c
int main(void) {
    printf("la reine s'éveille... ");   // pas de \n !
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    printf("[%s]\n", pid == 0 ? "fils" : "père");
    return 0;
}
```

1. Pourquoi « la reine s'éveille » apparaît-il **deux fois** ? (Le texte est encore dans le **tampon** de `printf`, en mémoire du processus, au moment du `fork`.)
2. Pourquoi le phénomène dépend-il de la redirection, même avec un `\n` ? (Vers un terminal, `stdout` est vidé à chaque ligne ; vers un fichier, seulement quand le tampon est plein.)
3. Corrigez avec `fflush(stdout);` juste avant le `fork`.

> **À RETENIR**
> Avant un `fork` ou un `exec`, appelez `fflush(stdout)` (ou `fflush(NULL)` pour tous les flux) : sinon ce qui est en attente dans les tampons de `stdio` est dupliqué (`fork`) ou perdu (`exec`).

Mission 2 — Vie et mort d'une ouvrière

EX 4 Qui meurt en premier ? (énoncé 1.2)

★ 10 min

Reprenez `famille.c` en retardant successivement la mort du père, puis celle du fils (primitive `sleep`). Dans chaque cas, ouvrez un second terminal et observez pendant l'attente :

```
TERMINAL
$ ps -o pid,ppid,stat,cmd -C famille
```

1. **Le père dort 20 s, le fils se termine aussitôt.** Quel est l'état (`STAT`) du fils pendant que le père dort ? Pourquoi le noyau garde-t-il une trace d'un processus terminé ?
2. **Le fils dort 5 s, le père se termine aussitôt.** Le fils affiche son PPID **après** son `sleep` : qui est devenu son père ? Le prompt du shell revient-il avant ou après le message du fils ? Pourquoi ?

> **INDICE**
> Un fils terminé dont le père n'a pas encore appelé `wait` est un **zombie** (état `Z`, `<defunct>`) : le noyau conserve son PID et son code de retour pour le père. Un fils dont le père est mort est **orphelin** : il est adopté par le PID 1 (`systemd`) ou par un processus « sous-faucheur » (*subreaper* : `systemd --user`, ou `/init` sous WSL).

EX 5 Recueillir l'ouvrière (énoncé 1.4)

★ 10 min

Synchronisez les processus avec `wait` pour que le père attende la mort de son fils et affiche alors le PID du fils mort. Puis transmettez une information par le **code de retour** :

```c
/* dans le fils */
exit(42);                       // code de retour : 0 à 255

/* dans le père */
int statut;
pid_t mort = wait(&statut);
if (mort == -1) { perror("wait"); exit(EXIT_FAILURE); }
if (WIFEXITED(statut))          // le fils s'est terminé par exit/return ?
    printf("ouvrière %d revenue avec le code %d\n", (int)mort, WEXITSTATUS(statut));
```

1. Essayez `exit(300)` dans le fils. Quel code reçoit le père ? Pourquoi ?
2. Remplacez l'`exit` par une division par zéro (ou `abort();`). Que vaut `WIFEXITED` ? Utilisez `WIFSIGNALED` et `WTERMSIG` pour afficher le numéro du signal (`kill -l` donne la correspondance).
3. Que renvoie un **second** appel à `wait` dans le père ? Affichez `errno` avec `perror`.

Drapeau : `FLAG{` + le code reçu par le père pour `exit(300)` + `}`.

EX 6 Le bourdonnement borné (énoncé 1.3)

★ 5 min

Programmez une boucle qui affiche **10 000 fois** le message d'identité, dans le père **et** dans le fils, après **un seul** `fork` (le `fork` est **avant** la boucle, jamais dedans). Redirigez la sortie vers un fichier et comptez les lignes de chacun : `./bzz > bzz.txt ; grep -c père bzz.txt`. Les messages sont-ils entrelacés ? Par blocs de quelle taille ? (Comparez avec un affichage vers le terminal.)

SÉCURITÉ La fork bomb : comprendre sans jamais la lancer

Une *fork bomb* est un programme qui crée des processus **sans limite** : chaque copie se duplique à son tour (`while (1) fork();`). En quelques secondes, la table des processus est pleine, plus personne ne peut lancer de commande, et la machine (ou le serveur partagé de la salle) doit être redémarrée. C'est un **déni de service**.

- On ne la lance **jamais**, même « pour voir » : c'est une faute, sur une machine partagée.
- **Protection** : la limite du nombre de processus par utilisateur. `ulimit -u` affiche la limite courante ; dans un **sous-shell**, `( ulimit -u 50 ; ./mon_prog )` empêche `mon_prog` et ses descendants de dépasser 50 processus : `fork` renvoie alors -1 (`EAGAIN`). Côté programme : `setrlimit(RLIMIT_NPROC, ...)` (voir `man 2 setrlimit`) ; côté système : `/etc/security/limits.conf` et les *cgroups* (`pids.max`).
- **Exercice** : lancez votre programme B de l'exercice 2 avec `( ulimit -u 3 ; ./b )`, observez les `fork: Resource temporarily unavailable` et vérifiez que votre test `fork() == -1` les traite proprement. (Si vous avez déjà d'autres processus, le plafond est atteint tout de suite : c'est le but.)

SÉCURITÉ Le zombie, une fuite de ressource

Un zombie n'occupe presque plus de mémoire, mais il occupe **une entrée de la table des processus** et un PID. Un serveur qui crée un fils par client sans jamais appeler `wait` accumule des zombies jusqu'à ne plus pouvoir créer de processus : c'est l'équivalent d'une **fuite mémoire** pour les processus. Règle : tout `fork` réussi doit avoir son `wait`/`waitpid`.

Mission 3 — L'essaim

EX 7 La reine et ses n ouvrières

★★ 10 min

Écrivez `ruche.c` (squelette dans `code/sujet06/ruche.c`) : la reine crée `N = 5` ouvrières. L'ouvrière numéro `i` dort `rand() % 4` secondes (graine différente : `srand(getpid())` **dans le fils**), affiche `ouvrière i (PID) : récolte terminée` et se termine avec `exit(i * 10)`.

1. La reine recueille ses ouvrières avec `wait` et affiche, pour chacune, son PID et son code. Dans quel ordre reviennent-elles ?
2. La reine mémorise les PID dans un tableau `pid_t ouvrieres[N]` et les recueille **dans l'ordre de création** avec `waitpid(ouvrieres[i], &statut, 0)`. Qu'est-ce qui change ?
3. Pourquoi faut-il appeler `srand` dans le fils et non avant les `fork` ?

PIÈGE CLASSIQUE L'ouvrière qui pond à son tour

Si le fils ne se termine pas par `exit` à la fin de son travail, il **revient dans la boucle** de la reine et crée lui-même des processus : 2^N processus au lieu de N. C'est l'erreur la plus fréquente de ce TP, et le début d'une fork bomb.

EX 8 Deux ouvrières, un seul rayon (énoncé 1.5)

★★ 10 min

Réalisez un programme qui lance deux processus. Le premier remplit le fichier `rayon.txt` avec 100 000 caractères `A`, le second avec 100 000 caractères `B`. Le père attend la fin des deux fils. Vérifiez le résultat : `wc -c rayon.txt`, `grep -o A rayon.txt | wc -l`, `head -c 300 rayon.txt`.

Testez **trois versions** et expliquez les différences :

Version	Ouverture du fichier
V1	`fopen("rayon.txt", "w")` **avant** le `fork` : les deux fils héritent du même flot.
V2	Chaque fils fait son propre `open("rayon.txt", O_WRONLY | O_CREAT, 0644)` **après** le `fork`.
V3	Comme V2 mais avec `O_WRONLY | O_CREAT | O_APPEND`.

INDICE

Après un `fork`, le fils hérite des **descripteurs** du père : ils partagent la même entrée de la table des fichiers ouverts, donc le **même offset**. Deux `open` séparés créent deux offsets indépendants, qui partent tous deux de 0 : les écritures s'écrasent. `O_APPEND` rend atomique le couple « aller à la fin + écrire ».

SÉCURITÉ Condition de course

Le résultat de V2 dépend de l'ordre d'exécution choisi par l'ordonnanceur : c'est une **condition de course** (*race condition*). Ce type de bogue est intermittent, donc difficile à reproduire, et peut devenir une faille (fichier de journal écrasé, données corrompues). Vous verrez les solutions (exclusion mutuelle) au sujet 07 et au sujet 08.

Mission 4 — La métamorphose : exec

`exec` **remplace** le code, les données et la pile du processus par ceux d'un autre programme. Le processus garde son **PID**, son père et ses descripteurs ouverts. En cas de succès, `exec` **ne revient jamais** : les instructions qui suivent ne s'exécutent que si `exec` a échoué.

Lettre	Signification
`l` (liste)	Arguments écrits un par un, terminés par `NULL` : `execlp("ls", "ls", "-l", NULL);`
`v` (vecteur)	Arguments dans un tableau terminé par `NULL` : `char *args[] = {"ls", "-l", NULL}; execvp("ls", args);`
`p` (PATH)	Le programme est cherché dans les répertoires de la variable `PATH`.
`e` (environnement)	On fournit soi-même les variables d'environnement (`execve`, `execle`).

EX 9 Changer de peau

★ 10 min

lance_ls.c

```c
int main(void)
{
    printf("avant exec ");
    execlp("ls", "ls", "-l", NULL);   // argv[0] = "ls", argv[1] = "-l"
    perror("execlp");                 // seulement si exec a échoué
    return 1;
}
```

1. « avant exec » s'affiche-t-il ? Pourquoi ? Corrigez (deux solutions).
2. Remplacez `"ls"` par `"lss"`. Que se passe-t-il ? Quel est le code de retour (`echo $?`) ?
3. Ajoutez un deuxième `execlp("date", "date", NULL);` après le premier. Est-il exécuté ? Dans quel cas le serait-il ?
4. Faites exécuter `ls -l` par un **fils** : le père attend la fin du fils puis affiche `commande terminée (code n)`. Vous venez d'écrire le cœur d'un shell : **fork, exec dans le fils, wait dans le père**.

SÉCURITÉ PATH piégé

`execlp` et `execvp` cherchent le programme dans `PATH`. Si `PATH` contient `.` ou un répertoire modifiable par un attaquant, un faux `ls` peut être exécuté à la place du vrai, **avec vos droits**. Un programme sensible (lancé par root, *setuid*, service) doit utiliser un **chemin absolu** (`execv("/usr/bin/ls", args)`) et un environnement maîtrisé (`execve`).

SÉCURITÉ Descripteurs hérités

Les descripteurs ouverts **survivent** à `fork` et à `exec` : le programme lancé hérite de vos fichiers ouverts (fichier de mots de passe, socket...). Ouvrez les fichiers sensibles avec le drapeau `O_CLOEXEC` (`open(chemin, O_RDONLY | O_CLOEXEC)`) pour qu'ils soient fermés automatiquement lors d'un `exec`.

Mission 5 — lamash, le shell du LAMAE

EX 10 Le mini-shell

★★ 25 min

Complétez `code/sujet06/lamash.c` (la lecture et le découpage de la ligne sont fournis). Le shell affiche une invite `lamash>` , lit une ligne avec `fgets`, la découpe en mots, puis :

1. si la ligne est vide, réaffiche l'invite ; si la commande est `exit`, quitte le shell ;
2. sinon crée un fils qui exécute la commande avec `execvp(mots[0], mots)` ; si `execvp` échoue, le fils affiche `lamash: commande introuvable` sur `stderr` et se termine avec `exit(127)` ;
3. le père attend **ce** fils avec `waitpid` et mémorise son code de retour ; la commande interne `status` l'affiche (équivalent de `echo $?`).

Testez : `ls -l`, `sleep 2`, `ps -l` (votre lamash apparaît comme père de `ps`), `false` puis `status`, `inexistant` puis `status`, Ctrl+D (fin de fichier : `fgets` renvoie `NULL`).

```
TERMINAL
$ ./lamash
lamash> ls -l /tmp
...
lamash> false
lamash> status
code de retour : 1
lamash> exit
```

SÉCURITÉ **Jamais system() avec une saisie utilisateur**

La solution de facilité serait `system(ligne)` : la ligne est alors interprétée par `/bin/sh`, avec ses métacaractères. Si un programme fait `system("ls " + nom)` avec un nom fourni par l'utilisateur, il suffit de saisir `x; rm -rf ~` pour exécuter une seconde commande : c'est une **injection de commande** (CWE-78). `popen` a le même défaut. Avec `execvp` et un tableau `argv`, chaque mot reste un **argument** : aucun shell n'interprète `;`, `|` ou `$( )`. Le guide ANSSI déconseille l'usage de `system`.

PIÈGE CLASSIQUE **La ligne et son '\n'**

`fgets` conserve le `'\n'` final : sans le retirer, `execvp` cherche une commande nommée `"ls\n"`. Le découpage fourni utilise `strtok` avec les séparateurs `" \t\n"`. Vérifiez aussi que le nombre de mots ne dépasse pas la taille du tableau `mots` (débordement !) et que le tableau se termine par `NULL`.

Drapeau : `FLAG{` + le code de retour affiché par `status` après une commande introuvable + `}`.

POUR LES RAPIDES **Améliorer lamash** ★★★

1. **Commande interne `cd`** : pourquoi `cd` ne peut-elle pas être un programme exécuté par un fils ? Implémentez-la avec `chdir` dans le père (et `getcwd` pour l'afficher dans l'invite).
2. **Arrière-plan** : si la ligne se termine par `&`, le père n'attend pas le fils et affiche `[PID]`. Les fils terminés deviennent zombies : vérifiez-le avec `ps`, puis ramassez-les avant chaque invite avec `while (waitpid(-1, NULL, WNOHANG) > 0)`.
3. **Chronomètre** : affichez la durée de chaque commande (`clock_gettime(CLOCK_MONOTONIC, ...)`).
4. **Signal** : affichez `tué par le signal n` quand `WIFSIGNALED` est vrai (testez `sleep 100` puis `kill` depuis un autre terminal).
5. **Traçage** : lancez `strace -f -e trace=clone,execve,wait4 ./lamash` et retrouvez les appels système de chaque commande (sous Linux, `fork` est implémenté par l'appel système `clone`).

Bilan de la séance

À RETENIR

- `fork` renvoie **0 au fils**, le **PID du fils au père**, **-1** en cas d'échec : on teste toujours les trois cas.
- Père et fils ont des **mémoires séparées** (copie à l'écriture) mais **partagent les fichiers ouverts** (même offset).
- n `fork` à la suite donnent 2^n processus : un fils doit se terminer par `exit`, et une boucle de `fork` est **toujours bornée**.
- `exit(code)` transmet un code de 0 à 255 ; le père le lit avec `wait`/`waitpid` puis `WIFEXITED` et `WEXITSTATUS`. Sans `wait`, le fils reste **zombie**.
- `exec` remplace le programme, garde le PID et les descripteurs, et **ne revient qu'en cas d'échec**. `fflush(stdout)` avant `fork` et `exec`.
- Le shell fait **fork + exec + wait** à chaque commande. Pour lancer une commande, `execvp` avec un tableau d'arguments, **jamais `system()`** avec une saisie.

- ☐ J'ai mes drapeaux dans `badge.txt` (missions 0 à 5).
- ☐ Aucun de mes programmes ne laisse de zombie (`ps -o stat` ne montre aucun `Z`).
- ☐ Mes programmes compilent sans **aucun** avertissement.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 2 fork`, `man 2 wait`, `man 2 waitpid`	Ce qui est hérité ou non, toutes les macros `W...`, l'option `WNOHANG`.
`man 2 execve`, `man 3 exec`	L'appel système et ses six variantes de la bibliothèque.
`man 7 credentials`, `man 2 setrlimit`	Identifiants (PID, PPID, UID, GID, sessions) ; limites de ressources (`RLIMIT_NPROC`).
`man ps`, `pstree`, `top` / `htop`, `man 5 proc`	Observer les processus ; `/proc/<pid>/status`, `fd/`, `maps`, `environ`.
`strace -f ./prog`	Voir tous les appels système, y compris ceux des fils (`-f`).
M. Kerrisk, *The Linux Programming Interface*, chap. 24 à 28	La référence : création, terminaison, surveillance des fils, exec, détails.
A. Tanenbaum, *Systèmes d'exploitation* (Pearson), chap. « Processus et threads »	Le modèle de processus, les états, l'ordonnancement.
J.-M. Rifflet, *La programmation sous UNIX*	Référence francophone, nombreux exemples de fork/exec.
Guide ANSSI, *Règles de programmation pour le développement sécurisé en C*	Vérification des retours, appels de commandes externes.

Pour approfondir

CULTURE **Copy-on-write et vfork**

Copier toute la mémoire du père à chaque `fork` serait lent : Linux partage toutes les pages en lecture seule et ne copie une page qu'au moment où l'un des deux processus l'écrit. Un `fork` suivi immédiatement d'un `exec` ne copie donc presque rien. `posix_spawn(3)` combine `fork` et `exec` en un seul appel, et c'est ce qu'utilisent de nombreux langages modernes pour lancer des commandes.

CULTURE **Les PID et le PID 1**

Les PID sont attribués de façon croissante jusqu'à `/proc/sys/kernel/pid_max`, puis recyclés. Le PID 1 (`systemd` ou `init`) est l'ancêtre de tous les processus ; il adopte les orphelins et appelle `wait` pour eux. Un processus peut se déclarer « sous-faucheur » (`prctl(PR_SET_CHILD_SUBREAPER)`) pour adopter les orphelins de sa descendance : c'est ce que font `systemd --user` et les gestionnaires de conteneurs.

CULTURE **Signaux et SIGCHLD**

Quand un fils se termine, le noyau envoie au père le signal `SIGCHLD`. Un serveur peut installer un gestionnaire (`sigaction`) qui appelle `waitpid(-1, NULL, WNOHANG)` en boucle pour ne jamais laisser de zombie, ou ignorer explicitement `SIGCHLD` pour que le noyau ne crée pas de zombies. Vous retrouverez ce mécanisme dans le serveur multi-clients du sujet 09.